

JavaScript ES6 介紹

ECMAScript 6 (簡稱 ES6)，是 JavaScript 語言新一代的標準，在 2015 年 6 月正式發佈。

ES6 其實是一個泛指的名詞，泛指 ES5.1 版以後的新一代 JavaScript 語言標準，涵蓋了 ES2015, ES2016, ES2017 等等，ES2015 則是正式名稱，特指該年度發佈的語言標準，現在常聽到人家說的 ES6，一般是指 ES2015 標準。

ES6 提出了很多新的語法，使得 JavaScript 變得更強大，更適合用來寫大型的應用程式！

這邊我會來介紹這些新的 ES6 特性：

1. Block Scope 程式區塊作用域 - let, const
2. Arrow Functions 箭頭函數
3. Default Function Parameters 函數預設參數
4. Spread/Rest Operator 展開/其餘運算子
5. Object Literal Extensions 物件實字擴展
6. Array and Object Destructuring 陣列和物件的解構賦值
7. Template Literals 模版字串
8. class 關鍵字

1. JavaScript ES6 Block Scope - let, const

在 ES6 之前，JavaScript：

用 **var** 關鍵字來宣告一個變數，其作用範圍是 **function scope**

在 ES6 中引入了兩個新的語法 **let** 和 **const** 讓你可以宣告程式區塊作用範圍 (block scope) 的變數。

所謂的 **block scope** 就是變數的作用範圍只存在兩個大括號 **{ }** 中。

let

let 關鍵字用來宣告一個 **block scope** 變數。

```
// global 變數
var a = 1;
{
  // block scope 變數
  let a = 2;

  // 2
  alert(a);

  // 重複宣告變數，會發生錯誤
  // TypeError: Identifier 'a' has already been declared
  let a = 3;
}
// 1
alert(a);
```

另外一個例子：

```
function varTest() {
  var x = 1;
  if (true) {
    // 同樣的 function scope 變數
    var x = 2;
    // 2
    alert(x);
  }
}
```

```
    // 2
    alert(x);
}
```

```
function letTest() {
    let x = 1;
    if (true) {
        // 不一樣的 block scope 變數
        let x = 2;
        // 2
        alert(x);
    }
    // 1
    alert(x);
}
```

const 常數

const 關鍵字跟 **let** 類似，也可以用來宣告一個 **block scope** 變數，不同的是，**const** 宣告的變數其指向的值不能再被改變。

```
{
    const A = 10;

    // 會發生錯誤，常數值不能再被改變
    // TypeError: Assignment to constant variable
    A = 10;

    // 陣列是一個有趣的例子
    const ARR = [1, 2];

    // 可以改變陣列裡的内容
    // 因為 ARR 變數值沒有改變，還是指向同一個陣列
    ARR.push(3);

    // [1, 2, 3]
    console.log(ARR);
}
```

```
// 錯誤，不能改常數值
// TypeError: Assignment to constant variable
ARR = 123;

// 但可以改變陣列裡面的內容
ARR[0] = 4;

// [4, 2, 3]
console.log(ARR);
}
```

const 還有一個要注意的地方，就是在宣告變數的同時就需要指定值，不然會發生 `SyntaxError` 錯誤，因為常數的值不能被更改！

```
// SyntaxError: Missing initializer in const declaration
const foo;
foo = 123;
```

```
// 正確的常數宣告方式必須同時賦值
const foo = 123;
```

2. JavaScript ES6 Arrow Functions 箭頭函數

ES6 允許我們用箭頭 `=>` 來更方便簡短的定義一個函數，這種語法稱作 `Arrow Functions`。

`Arrow Functions` 由 參數列 (...) 接著 `=>` 和函數內容所組成。

語法：

例子 1：

// 傳統標準的函數宣告語法

```
var multiply = function(a, b) {
  return a * b;
};
```

// 使用 `Arrow Functions` 的語法

```
var multiply = (a, b) => a * b;
```

```
// 20
multiply(2, 10)
```

例子 2：

```
let numbers = [1, 2, 3];
```

```
// callback 用 Arrow Functions 的寫法更精簡
```

```
let doubles = numbers.map(num => {
    return num * 2;
});
```

```
// [2, 4, 6]
```

```
console.log(doubles);
```

Arrow Functions 不僅只是提供一個更簡潔的語法，另外一個重要的特性是 Arrow Functions 會自動綁定 (bind) **this** 到宣告 Arrow Functions 當時的環境。

先來看傳統標準的函數宣告：

```
function Person() {
    this.age = 0;
    setInterval(function growUp() {
        // 這邊的 this 會指向 window，造成程式錯誤
        this.age++;
    }, 1000);
}
```

```
var p = new Person();
```

針對這問題，傳統的解法像是：

```
function Person() {
    // 先將正確的 this reference 存到一個變數中
    var self = this;
    self.age = 0;
    setInterval(function growUp() {
        // self 變數會正確的指向 Person 物件
        self.age++;
        document.getElementById('data').innerText = self.age;
    }, 1000);
}
```

Arrow Functions 最有用的特性之一，就是會自動綁定 `this` 到函數定義時的環境：

```
function Person() {  
    this.age = 0;  
    // 定義該 Arrow Functions 時的環境，是在 Person 物件中  
    setInterval(() => {  
        // 所以 this 會正確指向 Person 物件  
        this.age++;  
        document.getElementById('data').innerText = self.age;  
    }, 1000);  
}  
var p = new Person();
```

3. JavaScript ES6 Default Function Parameters

在 ES6 中，JavaScript 函數的參數終於可以有預設值了。

語法：

```
function funcName([param1[ = defaultValue1 ][, ..., paramN[ = defaultValueN ]]]) {  
    statements  
}
```

在參數名稱後面，接著等號 = 然後指定預設值。

傳統要給函數參數一個預設值，寫法會像是：

```
function multiply(a, b) {  
    b = (typeof b !== 'undefined') ? b : 1;  
    return a * b;  
}
```

```
multiply(5, 2); // 10  
multiply(5, 1); // 5  
multiply(5);    // 5
```

ES6 新的寫法簡潔多了：

```
function multiply(a, b = 1) {  
    return a * b;  
}
```

```
multiply(5, 2); // 10  
multiply(5, 1); // 5  
multiply(5); // 5
```

4. JavaScript ES6 Spread/Rest Operator 運算子

ES6 引入了新的運算子 **(operator) ... (三個點號)** 來表示展開或其餘運算子。

Spread Operator 展開運算子

Spread Operator 可以用在執行函數時的參數列上，它可以將一個陣列 (array) 展開，轉為多個逗點隔開的獨立參數：

```
function foo(x, y, z) {  
    console.log(x, y, z);  
}
```

```
var arr = [1, 2, 3];
```

```
// 輸出 1 2 3  
// 等同於執行 foo(1, 2, 3)  
foo(...arr);
```

也可以放在多個參數中間使用：

```
function foo(a, b, c, d, e) {  
    console.log(a, b, c, d, e);  
}
```

```
var arr = [3, 4];
```

```
// 輸出 1 2 3 4 5  
// 等同於執行 foo(1, 2, 3, 4, 5)  
foo(1, 2, ...arr, 5);
```

Spread Operator 也可以用在 array literal，用來展開陣列元素：

```
var parts = ['shoulders', 'knees'];
```

```
// 將 parts 展開在 lyrics 的元素中
```

```
var lyrics = ['head', ...parts, 'and', 'toes'];
```

```
// lyrics 的值會變成 ["head", "shoulders", "knees", "and", "toes"]
```

Spread Operator 也可以用來結合 (concat) 多個陣列：

```
var ary1 = [4, 5, 6];
```

```
var ary2 = [1, 2, 3];
```

```
// ary1 會變成 [1, 2, 3, 4, 5, 6]
```

```
ary1 = [...ary2, ...ary1];
```

Rest Operator 其餘運算子

在 ES6 之前，如果你想要一個函數可以接受不固定數量的參數，你可能會這樣寫：

```
function fn(a, b) {  
  var args = Array.prototype.slice.call(arguments, fn.length);
```

```
  // ...  
}
```

ES6 新的 Rest Operator 讓你可以更直觀的宣告不定長度參數。

上面的函數，用 Rest Operator 可以改寫為：

```
function fn(a, b, ...args) {  
  // ...  
}
```


`...args` 只能放在最後一個參數，用來獲取其餘的參數，`args` 的值是一個陣列 (array)，用來存放獲取的參數。

使用例子：

```
function fun1(...myArgs) {  
    console.log(myArgs);  
}
```

```
// 顯示 []  
fun1();
```

```
// 顯示 [1]  
fun1(1);
```

```
// 顯示 [5, 6, 7]  
fun1(5, 6, 7);
```

5. JavaScript ES6 Object Literal Extensions 物件實字的擴展

ES6 的新語法讓 `object literal` 可以寫得更簡短清楚。

Property value 物件屬性簡寫

如果你的屬性名稱和變數名稱一樣，你可以在 `object literal` 只使用變數，則變數的名稱會被當作是屬性名稱，而變數的值會被當作是屬性值。

用法：

```
var obj = {  
    foo,  
    bar  
};
```

上面的語法同等於：

```
var obj = {  
    foo: foo,  
    bar: bar  
};
```

Method definition 物件方法簡寫

除了物件的屬性可以簡寫外，物件的方法也可以。

ES6 新的寫法：

```
var obj = {  
  // 可以省略 function 關鍵字和冒號 (colon)  
  doSomething() {  
    // ...  
  }  
};
```

上面的寫法同等於：

```
var obj = {  
  doSomething: function() {  
    // ...  
  }  
};
```

Computed property keys 計算的屬性名稱

ES6 允許使用表達式 (expression) 作為屬性的名稱，語法是將 expression 放在中括號 [] 裡面，透過 [] 的語法，我們的屬性名稱就可以放入變數，達到動態產生屬性名稱的效果。

範例：

```
var prefix = 'es6';  
var obj = {  
  // 計算屬性  
  [prefix + ' is']: 'cool',  
  
  // 計算方法  
  [prefix + ' score']() {  
    console.log(100);  
  }  
};  
// 顯示 cool  
console.log(obj['es6 is']);  
// 顯示 100  
obj['es6 score']();
```

6. JavaScript ES6 Array and Object Destructuring 陣列和物件的解構賦值

ES6 的 **destructuring assignment**，可以用來從陣列或物件中解構 (destructuring) 值出來指定 (assignment) 給變數。

Array Destructuring 陣列的解構賦值

在 ES6 之前，賦值給一個變量，只能用指定的方式：

```
var one = 'one';  
var two = 'two';  
var three = 'three';
```

ES6 你可以用 **Destructuring** 的語法：

```
var foo = ['one', 'two', 'three'];
```

```
// 從 array 中提取值，然後按照對應的位置，指定給相應的變數  
var [one, two, three] = foo;
```

```
// "one"  
console.log(one);
```

```
// "two"  
console.log(two);
```

```
// "three"  
console.log(three);
```

另外一個使用的例子：

```
function foo() {  
    return [1, 2, 3];  
}
```

```
// [1, 2, 3]  
var arr = foo();
```

```
var [a, b, c] = foo();
```

```
// 顯示 1 2 3
console.log(a, b, c);
也可以有預設值 (default value) :
```

```
var [a=5, b=7] = [1];
```

```
// 1
console.log(a);
// 7
console.log(b);
可以留空來略過某些值 :
```

```
function f() {
    return [1, 2, 3];
}
```

```
// 跳過第二個值
var [a, , b] = f();
```

```
// 1
console.log(a);
```

```
// 3
console.log(b);
搭配 Rest Operator :
```

```
var [a, ...b] = [1, 2, 3];
```

```
// 1
console.log(a);
```

```
// [2, 3]
console.log(b);
```

Object Destructuring 物件的解構賦值

Destructuring 不僅可以用在 array，也可以用在 object：

```
var o = {p: 42, q: true};
```

```
// 從物件 o 中取出 key 為 p 和 q 的值，指定給變數 p 和 q  
var {p, q} = o;
```

```
// 42  
console.log(p);
```

```
// true  
console.log(q);
```

也可以取值給已經宣告過的變數：

```
var a, b;
```

// 小括號 () 是必要的，不然左邊的 {} 會 `SyntaxError` 被當作是 block 區塊宣告

```
{a, b} = {a: 1, b: 2};
```

你的變數名稱可以和被取值的 object key 使用不同的名稱：

```
var o = {p: 42, q: true};
```

```
// 取出 key 名稱為 p 的值，指定給變數 foo  
// 取出 key 名稱為 q 的值，指定給變數 bar  
var {p: foo, q: bar} = o;
```

```
// 42  
console.log(foo);
```

```
// true  
console.log(bar);
```

也可以有預設值 (default value)：

```
var {a = 10, b = 5} = {a: 3};
```

```
// 3
```

```
console.log(a);
```

```
// 5
```

```
console.log(b);
```

其實只要兩邊的結構 (patterns) 一樣，左邊的變數就會被賦予對應的值，所以用在 nested object 或 array 也行：

```
var obj = {  
  p: [  
    'Hello',  
    {  
      y: 'World'  
    }  
  ]  
};
```

```
var {p: [x, {y}]} = obj;
```

```
// "Hello"
```

```
console.log(x);
```

```
// "World"
```

```
console.log(y);
```

還可以用在函數的參數列上：

```
function drawChart({size = 'big', cords = {x: 0, y: 0}, radius = 25} = {}) {  
  console.log(size, cords, radius);  
}
```

```
// 輸出 big {x: 18, y: 30} 30
```

```
drawESChart({  
  cords: {x: 18, y: 30},  
  radius: 30  
});
```

7. JavaScript ES6 class 關鍵字

在 ES6 中，引入了 **Class** (類別) 這個新的概念 (如果寫過 C++ 或 Java 等傳統語言應該非常熟悉)，透過 **class** 這新的關鍵字，可以定義類別。

另外還引入了一些其他的新語法，來讓你更簡單直觀的用 JavaScript 寫 OOP (物件導向) 程式，並不是重新設計一套物件繼承模型 (object-oriented inheritance model)，只是讓你更方便操作 JavaScript 既有的原型繼承模型 (prototype-based inheritance)。

在 ES6 你可以用 **class** 語法定義一個類別：

```
class Animal {  
  constructor(name) {  
    this.name = name;  
  }  
  
  speak() {  
    console.log(this.name + ' makes a noise.');  }  
}
```

上面我們定義了一個 **Animal** 類別：

其中 **constructor** 方法用來定義類別的建構子 (constructor) 方法 (method) 的定義也有新語法，我們在 **Animal** 類別中定義了 **speak()** 方法，你可以看到新語法省略了 **function** 關鍵字和冒號：
我們說過新語法只是語法糖，底層還是 **prototype-based** 的關係：

```
let a = new Animal('Elephant');
```



```
// true  
a.constructor === Animal.prototype.constructor;
```



```
// true  
a.speak === Animal.prototype.speak;
```



```
// true  
Animal.prototype.constructor === Animal;
```

```
// "function"
```

`extends` 關鍵字用作類別繼承：

```
class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(this.name + ' makes a noise.');
```

```
  }
}

class Dog extends Animal {
  speak() {
    console.log(this.name + ' barks.');
```

```
  }
}

var d = new Dog('Mitzie');
```

```
// 顯示 Mitzie barks.
```

```
d.speak();
```

如果子類別 (sub-class) 有定義自己的 `constructor`，必須在 `constructor` 方法中顯示地調用 `super()`，來調用父類別的 `constructor`，否則會出現錯誤 -

`ReferenceError: this is not defined`。

而且在 `sub-class constructor` 中，必須先執行完 `super()` 後，才能引用 `this` 關鍵字，否則也會出現錯誤 - `ReferenceError: this is not defined`。

這是因為在 ES6 中，是先建立父類別 (parent class) 的物件實例 `this` (所以必須先執行 `super()`)，然後再用子類別的 `constructor` 修改 `this`。

```
class Car {
  constructor() {
    console.log('Creating a new car');
```



```

    }
}

class Porsche extends Car {
    constructor() {
        super();
        console.log('Creating Porsche');
    }
}

```

```
let c = new Porsche();
```

```

// 依序顯示
// Creating a new car
// Creating Porsche
super 關鍵字有兩種用法：

```

當作函數 `super()`，只能在子類別的 `constructor` 中使用，在其他地方用會報錯 -

`SyntaxError: 'super' keyword unexpected here`

當作物件 `super`，在一般方法中使用，用來引用父類別的方法和屬性

```

class Cat {
    constructor(name) {
        this.name = name;
    }

    speak() {
        console.log(this.name + ' makes a noise.');
```

```

    }
}

class Lion extends Cat {
    speak() {
        super.speak();
        console.log(this.name + ' roars.');
```

```

    }
}

```

```
let bigCat = new Lion('Hoo');
```

```
bigCat.speak();  
// 依序顯示  
// Hoo makes a noise.  
// Hoo roars.
```

另外，透過 `super` 調用父類別的方法時，`super` 會綁定子類別的 `this` (而不是父類別的 `this`)：

```
class A {  
    constructor() {  
        this.x = 1;  
    }  
  
    print() {  
        console.log(this.x);  
    }  
}
```

```
class B extends A {  
    constructor() {  
        super();  
        this.x = 2;  
    }  
  
    foo() {  
        super.print();  
    }  
}
```

```
let b = new B();
```

```
// 顯示 2 而不是 1
```

```
b.foo();
```

```
static
```

`static` 關鍵字用來定義靜態方法 (static method)。

```
class StaticMethodCall {  
    static staticMethod() {
```

```
        return 'Static method has been called';
    }

    static anotherStaticMethod() {
        // 你可以用 this 來調用其他的 static method
        return this.staticMethod() + ' from another static method';
    }
}
```

```
// 顯示 Static method has been called
StaticMethodCall.staticMethod();
```

```
// 顯示 Static method has been called from another static method
StaticMethodCall.anotherStaticMethod();
```

父類別上的靜態方法也可以透過 `super` 來調用：

```
class Triple {
    static triple(n) {
        if (n === undefined) {
            n = 1;
        }
        return n * 3;
    }
}

class BiggerTriple extends Triple {
    static triple(n) {
        return super.triple(n) * super.triple(n);
    }
}
```

```
// 3
console.log(Triple.triple());
```

```
// 18
console.log(Triple.triple(6));
```

```
var tp = new Triple();
```

```
// 81
console.log(BiggerTriple.triple(3));

// 報錯
// TypeError: tp.triple is not a function
console.log(tp.triple());
```

8. JavaScript ES6 Template Literals 字串樣版

ES6 引入了 Template Literals (字串樣版) 是增強版的字串表示法，Template Literals 讓你可以寫多行字串 (multi-line strings)、也可以在字串中插入變數或 JavaScript 表達式 (String/Expression interpolation)。

Template Literals 的語法是用兩個反引號 (back-tick) `` 標示，而在字串中可以使用 \${} 語法來嵌入變數或 JavaScript 表達式。

多行字串 Multi-line Strings
傳統的寫法：

```
var str = 'string text line 1\n' +
'string text line 2'
```

Template Literals 新的寫法：

```
var str = `string text line 1
string text line 2`
```

嵌入變數或任何表達式 String/Expression Interpolation
傳統的寫法：

```
var a = 5;
var b = 10;
console.log('Fifteen is ' + (a + b) + ' and\nnot ' + (2 * a + b) + '.');
Template Literals 新的寫法：
```

```
var a = 5;
var b = 10;
console.log(`Fifteen is ${a + b} and
not ${2 * a + b}.`);
```

`${}` 中可以是任何 JavaScript expression，例如執行一個函數：

```
function fn() {  
    return 'Hello World';  
}
```

```
// foo Hello World bar  
var str = `foo ${fn()} bar`;
```

樣板標籤 Tagged Template Literals

Template Literals 可以接在一個函數名稱後面，該函數會被用來處理字串樣版，這功能叫做樣板標籤 (Tagged Template)。

如果字串中沒有使用 `${}` 就跟執行一般函數類似，執行函數時，會傳入一個陣列型態的參數，陣列中只有一個元素就是字串本身：

```
alert`123`;
```

```
// 等同於執行
```

```
alert(['123']);
```

如果字串中有使用 `${}`，會將字串樣版的內容切分成多個參數，切法是先根據 `${}` 的位置，將沒有在 `${}` 中的內容切開成多個字串組成一個陣列 (array)，當作函數的第一個參數，接著將每個 `${}` 中的值，依序當作第二個、第三個、...第 N 個參數：

```
var person = 'Mike';  
var age = 28;
```

```
function myTag(strings, personExp, ageExp) {
```

```
    // 首先依 ${} 的位置將原始字串切成一個字串陣列，得到 strings  
    // ["that ", " is a ", ""]  
    //  
    // 為什麼最後有一個 ""  
    // 因為我們有一個 ${} 的位置在字串結尾  
  
    // ${ person } 的值會被當作第二個參數傳入  
    // ${ age } 的值會被當作第三個參數傳入
```

```

// "that "
var str0 = strings[0];
// " is a "
var str1 = strings[1];

var ageStr;

if (ageExp > 99) {
    ageStr = 'centenarian';
} else {
    ageStr = 'youngster';
}

// ${ person } 的值會被當作第二個參數傳入
return str0 + personExp + str1 + ageStr;
}

var output = myTag`that ${ person } is a ${ age }`;

// 顯示 that Mike is a youngster
console.log(output);

const visitMessage = (cus) => {
    // 物件解構
    const { age, name } = cus;
    // 字串模板
    const text = age < 18 ?
        `Dear ${name}, you are under age:${age}`
        : `Welcome, ${name}:${age}!`;
    return text;
};

var txt = visitMessage({ "name": "Mary", "age": 21 });
alert(txt);

```